MUSEUM HEIST

A Grid-Based Stealth Game Integrating Greedy Search, Backtracking, Linked Lists, and Binary Search Trees

Nelson de Jesus Navarro De La Rosa · Henry Samuel Garrido Medina · Josue Urrego Lopez

Computer Science I · Systems Engineering Program
Universidad Distrital Francisco José de Caldas · Bogotá, Colombia
2026-I

Abstract — Museum Heist is a Computer Science I final project implemented as a fully playable grid-based stealth game. The system integrates Python, Pygame, C++17, and JSON file-based inter-process communication to demonstrate core course concepts through interactive mechanics rather than isolated code examples. The player navigates a museum floor plan, collects valuable items, evades camera and guard vision zones, and reaches an exit before losing by alarm, movement limit, or time limit. A C++ rule engine validates every action and owns the authoritative game state; Python renders the graphical interface, executes a greedy item recommendation, and computes a backtracking escape route. Custom C++ data structures — a singly linked list for movement history and a binary search tree ordered by item value — satisfy the course data-structure requirements. Three difficulty levels, five screen states, a greedy recommendation, a real-time backtracking route, and a full rank system combine to produce a project that is both technically sound and engaging to play. All 18 planned test cases passed on the final build.

Index Terms *— greedy algorithm, backtracking, linked list, binary search tree, Pygame, C++17, JSON inter-process communication, grid-based stealth game.*

TABLE OF CONTENTS

I. INTRODUCTION

Algorithms and data structures are most effectively understood when they drive visible, interactive behaviour rather than serving as abstract examples. Museum Heist was designed with this principle in mind: every required course concept — a greedy heuristic, a backtracking search, a linked list, and a binary search tree (BST) — maps directly to a mechanic the player can see and feel during gameplay.

The game places the player in a museum grid. Guards occupy fixed positions and project vision cones in all four cardinal directions. Cameras project vision in a single fixed direction. Items of varying monetary value are scattered across the floor. The player must reach the exit cell while optionally collecting items, keeping an alarm counter below the per-difficulty maximum, staying within a movement budget, and completing the run before the clock expires.

The project is organised as a two-process system: a compiled C++ binary handles all rule logic and produces a canonical state file; a Python/Pygame front-end reads that file, runs the two Python algorithms, and renders everything to the screen. Communication is entirely file-based through two JSON files, making the data flow observable and debuggable.

II. REQUIREMENTS AND SCOPE

Table I maps each course requirement to the file and structure that satisfies it. Every graded concept is exercised by a live, player-visible mechanic.

Requirement	Implementation
Greedy algorithm	greedy.py: selects the single highest-priority uncollected item each turn based on proximity to guards and vision-zone safety.
Backtracking	backtracking.py: depth-first search from player to exit avoiding all blocked cells; canonical choose / explore / unchoose pattern.
Linked list	MovementList in linked_list.h/.cpp: custom singly linked list; O(1) push_back via tail pointer.

Requirement	Implementation
Binary search tree	ItemBST in tree.h/.cpp: custom BST keyed by item value; inorder traversal produces the sorted item array in state.json.
JSON bridge	input.json carries action + state to C++; state.json returns full updated state to Python.
Pygame UI	main.py: renders five screens, board, HUD, vision overlays, backtracking route, and greedy highlight.

Table I. Course Requirement Coverage

III. ARCHITECTURE

A. Directory Layout

The repository is split into two primary modules — engine/ (C++17) and game/ (Python/Pygame) — connected by two JSON files stored in game/data/:

```
museum-heist/
  engine/
    main.cpp            # C++ rule engine (1,106 lines)
    linked_list.h/.cpp  # custom singly linked list
    tree.h/.cpp         # custom binary search tree
  game/
    algorithms/
      greedy.py         # greedy item selector
      backtracking.py   # backtracking escape router
    data/
      input.json        # Python -> C++ channel
      state.json        # C++ -> Python channel
    ui/
      main.py           # Pygame renderer + event loop
      bridge.py         # subprocess + JSON I/O
    requirements.txt    # pygame only
```

B. Execution Cycle

Each player action triggers one full deterministic round-trip:

- **1. Event capture.** Pygame detects a keyboard event (KEYDOWN) or a timer tick.
- **2. Serialise.** bridge.write_input() writes input.json with the action, direction, difficulty, and all state fields C++ must not lose (movement history, collected items, score, alarm, elapsed seconds).
- **3. Engine run.** bridge.run_engine() invokes the compiled binary via subprocess.run().
- **4. Deserialise.** bridge.load_state() reads state.json and returns a Python dict.
- **5. Algorithm pass.** choose_greedy_item(state) and find_escape_route(state) are called; results feed the renderer.
- **6. Render.** Pygame draws the updated board, vision overlays, route, greedy highlight, and HUD.

C. Design Rationale

File-based IPC was chosen for explainability: any state snapshot is human-readable JSON and can be inspected in a text editor. Python was kept as the UI and algorithm layer because Pygame provides rendering primitives and dynamic typing shortens algorithm prototyping. C++ was used for the rule engine to fulfil the course requirement of implementing custom data structures in a compiled, memory-managed language.

IV. C++ RULE ENGINE

The engine is the single source of truth for game state. It is compiled with:

```
g++ -std=c++17 engine/main.cpp engine/linked_list.cpp \
    engine/tree.cpp -o engine/museum_engine
```

A. Core Structs

Five plain structs are defined; no STL containers are used for game logic. Fixed-size arrays with compile-time constants (MAX_WALLS=16, MAX_CAMERAS=3, etc.) replace std::vector deliberately, keeping the memory layout predictable:

```
struct Position  { int row; int col; };
struct ItemData  { string id; int row,col,value; };
struct CameraData{ string id,direction; int row,col,range; };
struct GuardData { string id; int row,col,vision_range; };
struct MapData   { string difficulty; int rows,cols,
                   max_alarm,max_movements,time_limit,...;
                   Position start,exit;
                   Position    walls [MAX_WALLS];
                   CameraData cameras[MAX_CAMERAS];
                   GuardData   guards [MAX_GUARDS];
                   ItemData    items  [MAX_ITEMS]; };
```

B. Difficulty Parameters

Difficulty	Grid	Max Alarm	Max Moves	Time Limit	Items
Easy	8×8	4	45	90 s	3
Normal	8×8	3	35	60 s	3
Hard	10×10	2	28	45 s	4

Table II. Difficulty Parameters

C. Movement Validation Pipeline

apply_input() is the central function of the engine. After reconstructing state from input.json, it applies the following ten-step validation chain for each 'move' action:

- Reconstruct state from JSON (history, collected items, score, alarm, elapsed).
- Time-limit check: if elapsed ≥ time_limit → game_over, reason = "Time expired".
- Compute next cell from direction string.
- Out-of-bounds: silently ignored — player stays in place.
- Wall collision: alarm++ then check alarm limit.
- Camera / guard / vision cell: instant game_over, reason = "Caught in vision zone".
- Valid move: update player position and push node to MovementList.
- Adjacency alarm: if player is immediately adjacent to a guard, alarm++.
- Item collection: score += value; near-guard penalty (Manhattan dist ≤ 2) costs one alarm.
- Escape / movement-limit checks: set escaped or game_over accordingly.

D. Vision Zone Computation

Camera vision propagates in one direction until it hits a wall or the grid boundary. Guard vision propagates in all four cardinal directions up to vision_range. The resulting vision_zones array is written to state.json and consumed by both the Pygame renderer (shaded overlay) and the Python algorithms (safe/dangerous cell classification).

E. Rank System

After a successful escape, calculate_rank() evaluates four criteria:

Rank	Score	Moves	Time	Alarm
A	≥ 70% of total item value	≤ 50% of max	≤ 50% of limit	≤ 1
B	≥ 40% of total item value	≤ 100% of max	≤ 100% of limit	Any
C	< 40% of total item value	Any	Any	Any
Caught	— (game_over = true)	—	—	—

Table III. Rank Calculation Criteria

V. JSON BRIDGE (BRIDGE.PY)

bridge.py encapsulates the entire inter-process contract in five functions. Key design decisions:

- write_input() converts any 'move' action to 'none' when the game is already over, preventing the engine from processing stale events.
- compile_engine() is called only once at startup via get_game_state(), not on every move.
- All paths are resolved relative to PROJECT_ROOT using pathlib.Path, making the project relocatable.
- elapsed_seconds is forwarded on every tick action so the C++ engine can enforce the time limit without an internal clock.

Field	Direction	Type / Values
action	Python → C++	"move" | "reset" | "tick" | "none"
direction	Python → C++	"up" | "down" | "left" | "right" | ""
difficulty	Python → C++	"Easy" | "Normal" | "Hard"
player	Both	{row, col}
movement_history	Both	array of {step, row, col}
collected_items	Both	array of item id strings
score / alarm	Both	integers
escaped / game_over	Both	booleans
vision_zones	C++ → Python	array of {source, type, row, col}
items_sorted_by_value	C++ → Python	BST inorder output — ascending by value
status / rank	C++ → Python	"Playing"|"Escaped"|"Caught" / A|B|C
elapsed_seconds	Both	integer — Python tracks, C++ enforces

Table IV. JSON File Schemas (input.json and state.json)

VI. CUSTOM C++ DATA STRUCTURES

Neither std::list nor std::map/std::set is used. Both structures are implemented from scratch to demonstrate the underlying pointer and node mechanics at the level required by the course.

A. MovementList — Singly Linked List

1) Interface

```
struct MovementNode {
    int row, col, step;
    MovementNode* next;
};

class MovementList {
    MovementNode* head;
    MovementNode* tail;   // enables O(1) append
    int size;
public:
    void push_back(int row, int col, int step); // O(1)
    void clear();                               // O(n)
    void write_json_array(ostream& out) const;  // O(n)
    int  get_size() const;                      // O(1)
};
```

2) Complexity

Operation	Time	Notes
push_back	O(1)	tail pointer avoids full traversal
clear	O(n)	must free every allocated node
write_json_array	O(n)	single traversal from head
get_size	O(1)	counter maintained on every push_back

Table V. MovementList Complexity

3) Design Note

The tail pointer is the critical optimisation: without it, each append would require traversing the entire list (O(n)). Because every valid move appends exactly one node, the total cost of tracking movement history across a full game run is O(m) where m is the movement count — rather than $O(m^2)$.

B. ItemBST — Binary Search Tree

1) Interface

```
struct ItemNode {
    string id;
    int row, col, value;
    ItemNode* left;
    ItemNode* right;
};

class ItemBST {
    ItemNode* root;
public:
```

```
    void insert(const string& id, int row, int col, int value); // O(log n) avg
    void inorder_json(ostream& out) const;                       // O(n)
    void clear();                                                // O(n)
    int  get_size() const;                                       // O(1)
};
```

2) Ordering Invariant

Smaller values go left; larger values go right. Equal values use the item id string as a deterministic tie-breaker (lexicographically smaller id goes left), guaranteeing consistent ordering even for maps with repeated values.

3) Complexity

Operation	Average Case	Worst Case
insert	O(log n)	O(n) — degenerate (right-skewed) tree
inorder_json	O(n)	O(n)
clear	O(n)	O(n)
get_size	O(1)	O(1)

Table VI. ItemBST Complexity

In practice n ≤ 4 (Hard difficulty), so worst-case behaviour is irrelevant for game performance; the BST is included to satisfy the course requirement and to make the connection between tree structure and sorted output visually concrete in the sidebar HUD.

VII. PYTHON ALGORITHMS

A. Greedy Item Selector (greedy.py)

1) Purpose

choose_greedy_item(state) selects exactly one uncollected item per turn to highlight in the UI. It is greedy because it commits to the locally best choice — based on value and immediate danger assessment — without simulating future moves or considering the overall route.

2) Decision Hierarchy

- **Priority 1 — High-value item near a guard, currently safe:** Items within Manhattan distance ≤ 2 of any guard trigger an alarm on pickup. Prioritising them while they are outside the vision zone maximises expected score per alarm unit spent.
- **Priority 2 — Highest-value safe item:** If no near-guard safe item exists, the algorithm selects the uncollected item with the highest value that lies outside all vision zones.
- **Priority 3 — Warning fallback:** If every remaining item is inside a vision zone, the highest-value item is returned with a warning reason string.

3) Complexity

Let i = uncollected items, g = guards, v = vision-zone cells. Each item is checked against all guards O(g) and all vision cells O(v), giving O(i(g + v)) overall. With i ≤ 4, g ≤ 3, and v ≤ 18 in Hard mode, the function runs in effectively constant time.

B. Backtracking Escape Router (backtracking.py)

1) Purpose

find_escape_route(state) performs a depth-first search from the player's current cell to the exit. It follows the canonical choose – explore – unchoose backtracking pattern: a cell is added to the route before recursing and removed if that branch fails.

2) Blocked-Cell Classification

A cell is blocked if it is:

- A wall.
- A camera or guard position.
- A vision-zone cell (camera or guard projection).
- Any cell already in the current recursion path (cycle prevention).

3) Move Ordering Heuristic

Candidate moves are sorted by Manhattan distance to the exit before recursing. This heuristic guides the search toward the goal first, dramatically reducing nodes expanded in typical game states and making the first path found nearly optimal.

4) Complexity

Theoretical worst case is $O(4^n)$ where n is the number of grid cells (up to 100 in Hard mode). In practice, walls and vision zones partition the grid significantly, and the Manhattan-distance heuristic makes full expansion rare. The route is recomputed after every player move.

VIII. PYGAME USER INTERFACE (MAIN.PY)

A. Window Layout

The board occupies the left portion (TILE_SIZE = 54 px per cell, MARGIN = 28 px). A sidebar of 340 px to the right contains the title, status banner, live metrics HUD, greedy recommendation, route length, and keyboard shortcuts.

B. Screen State Machine

The UI implements a lightweight state machine with five states:

State	Description / Transitions
start	Difficulty selector. Keys 1/2/3 change difficulty; Enter starts; H opens tutorial.
tutorial	Static help screen. B returns; Enter starts game; R resets.
playing	Active gameplay. Arrow/WASD move; H pauses to tutorial; R resets; M returns to menu.
escaped	Victory screen with final metrics. R or M available.
caught	Defeat screen with game-over reason. R or M available.

Table VII. Screen States and Transitions

C. Rendering Pipeline

Each frame in the playing state renders in the following order to ensure correct visual layering:

- Floor tiles — checkerboard in two alternating beige tones.

- Vision-zone overlay — semi-transparent surface over each vision cell.
- Backtracking route — small dots at each route cell centre.
- Walls — dark rectangles with border.
- Items — diamond polygons.
- Greedy outline — pulsing rectangle border around the selected item (≈ 180 ms period).
- Exit — filled rectangle.
- Cameras — triangle polygon.
- Guards — filled circle.
- Player — filled circle (distinct colour).

D. Sidebar HUD

Nine live metrics are displayed: Status, Difficulty, Score, Items collected/total, Moves/max, Time/limit, Alarm/max, Route length, and Rank. The alarm value changes colour when approaching or reaching the limit. The greedy-selected item and its reason string appear below the metrics.

IX. TESTING AND VALIDATION

A. Test Plan

Table VIII lists all 18 manual test cases executed against the final build. Each case verifies one specific engine or algorithm behaviour. All 18 passed.

ID	Component	Expected Result
T01	Engine build	Compiles without warnings or errors.
T02	Pygame launch	Window opens at correct size.
T03	Valid move	Player advances one cell; history grows.
T04	Wall collision	Player stays; alarm increments by 1.
T05	Out-of-bounds	Player stays; no alarm change.
T06	Item pickup	Score increases by item value; item disappears.
T07	Near-guard pickup	Alarm increments on collection.
T08	Vision zone	Immediate game-over; reason = "Caught in vision zone".
T09	Alarm limit	Game-over when alarm ≥ max_alarm.
T10	Move limit	Game-over when history length ≥ max_movements.
T11	Time limit	Game-over on tick when elapsed ≥ time_limit.
T12	Escape	status = "Escaped"; victory screen shown.
T13	Rank A	Score ≥ 70%, half moves, half time, alarm ≤ 1.
T14	Restart (R)	All metrics reset; map re-loaded.
T15	Main menu (M)	Returns to start screen without error.

ID	Component	Expected Result
T16	Greedy output	Returns correct item id and reason string.
T17	Backtracking	Returns valid non-colliding route or empty list.
T18	BST inorder	items_sorted_by_value ascending by value.

Table VIII. Complete Validation Test Suite

B. Hard-Mode State Verification

The state.json produced by a Hard-mode reset was inspected to confirm the BST inorder output. With items coin (25), vase (60), mask (80), and painting (100), the BST produces the expected ascending sequence:

```
[
  {"id":"coin",     "row":8,"col":3,"value":25},
  {"id":"vase",     "row":6,"col":1,"value":60},
  {"id":"mask",     "row":1,"col":8,"value":80},
  {"id":"painting", "row":2,"col":6,"value":100}
]
```

X. DISCUSSION

A. Algorithms as Game Mechanics

The greedy algorithm gives the player an immediate, visible recommendation each turn, making the concept of local optimality without global guarantee tangible: the highlighted item is not always the one that maximises the final score, because it ignores route cost and future alarm accumulation. The backtracking route changes in real time as the player moves and vision zones shift, demonstrating the choose-explore-unchoose pattern dynamically.

The linked list's tail-pointer optimisation (O(1) append) is demonstrated live with every move. The BST's inorder output drives the sidebar 'items by value' list automatically, making the connection between tree structure and sorted output concrete for the player.

B. Limitations

- **Hardcoded maps.** All three maps are compiled into the binary. Adding a map requires recompilation, limiting replay value.
- **Manual JSON parser.** The get_string_value / get_int_value helpers are fragile against unexpected whitespace or nested keys.
- **Unbalanced BST.** Inserting items in ascending value order produces a right-skewed tree with O(n) insert in the worst case. An AVL or red-black tree would guarantee O(log n).
- **Backtracking does not optimise score.** The route found is the first feasible path to the exit, not the one that passes through the most valuable items.
- **Static guards.** Guards do not patrol; their positions are fixed for the entire run, reducing strategic depth.

C. Future Work

- Procedural map generation with a configurable seed.
- Replace the manual JSON parser with nlohmann/json for robustness.
- Implement an AVL tree to eliminate worst-case O(n) insert.
- Add guard patrol paths with periodic vision recalculation.

- Extend backtracking to maximise collected item value along the route (branch-and-bound or A*).

XI. CONCLUSION

Museum Heist successfully integrates C++17, Python 3, Pygame, and JSON inter-process communication into a playable stealth game where each course concept is a live, observable mechanic. The C++ engine provides a compiled, memory-managed backend with a custom linked list and BST; the Python layer provides a high-productivity environment for rendering and algorithm prototyping. The file-based IPC makes the data flow transparent and debuggable.

The architecture cleanly separates responsibilities: C++ owns rules and data structures, Python owns interaction and visualisation, and JSON connects both layers. All 18 planned test cases passed on the final build, validating the correctness of each individual component and the integration as a whole. The project demonstrates that algorithms and data structures can drive a fully playable system rather than remain isolated exercises.

REFERENCES

[1] C. A. Sierra, *Computer Sciences I: Course Project Description*, Universidad Distrital Francisco Jose de Caldas, 2026-I.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.

[3] S. S. Skiena, *The Algorithm Design Manual*, 2nd ed. Springer, 2008.

[4] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley, 2011.

[5] Pygame Community, *Pygame Documentation*, 2024. [Online]. Available: https://www.pygame.org/docs/

[6] ISO/IEC, *Programming Languages — C++*, ISO/IEC 14882:2017 (C++17), 2017.

APPENDICES

Appendix A — Game Controls

Key(s)	Action
Arrow keys / WASD	Move player one cell in that direction
Enter	Start game from current screen
R	Restart current difficulty (full reset)
M	Return to main menu
H	Toggle tutorial / help screen
1 / 2 / 3	Select Easy / Normal / Hard (on start screen)

Table A.1. Keyboard Controls

Appendix B — Win and Loss Conditions

Condition	Type	Trigger
Reach exit cell	Win	player position == exit position
Enter vision zone	Loss	player steps onto a camera or guard vision cell
Alarm limit reached	Loss	alarm ≥ max_alarm (wall hits, guard proximity, risky items)
Movement limit	Loss	history.size() ≥ max_movements
Time limit	Loss	elapsed_seconds ≥ time_limit (enforced by tick action)

Table A.2. Win and Loss Conditions

GLOSSARY

Backtracking — A recursive search strategy that builds a solution incrementally and abandons partial solutions as soon as they cannot lead to a valid result (choose – explore – unchoose).

Binary Search Tree (BST) — A binary tree where each node's left subtree contains smaller keys and the right subtree contains larger keys, enabling O(log n) average search and insertion.

Greedy Algorithm — An algorithm that makes the locally optimal choice at each step without reconsidering previous decisions. Does not guarantee a globally optimal solution.

HUD (Heads-Up Display) — The on-screen overlay showing live game metrics: score, moves, alarm, time, rank, and algorithm outputs.

IPC (Inter-Process Communication) — Mechanisms allowing separate processes to exchange data. Here, file-based IPC via JSON files is used between the C++ engine and Python frontend.

Manhattan Distance — The sum of absolute differences in row and column coordinates between two cells: |r1−r2| + |c1−c2|.

Vision Zone — A set of grid cells reachable by a camera or guard's line of sight. Entering a vision zone triggers an immediate game-over.